

AIM

To develop an Android application to perform basic mathematical operations (addition, subtraction, multiplication, and division) using the Android Studio Integrated Development Environment.

APPARATUS REQUIRED

- Computer
- Android Studio software

THEORY

Introduction to Android

Android is an open-source mobile operating system based on a modified version of the Linux kernel and other open-source software, designed primarily for touchscreen mobile devices such as smartphones and tablets. Android applications are written predominantly in Java or Kotlin programming languages and are compiled into bytecode for the Android Runtime (ART), which executes the applications on Android devices.

Android Application Architecture

The Android application architecture consists of several key components that work together to create a functional mobile application. The primary components include Activities, Services, Broadcast Receivers, and Content Providers. For this mathematical operations application, we primarily utilize the Activity component, which represents a single screen with a user interface.

Application Overview

The Mathematical Operations application serves as a fundamental exercise in understanding Android application development principles. This application demonstrates the implementation of basic user interface elements, event handling mechanisms, and mathematical computations. The application allows users to input two numerical values and perform four basic arithmetic operations: addition, subtraction, multiplication, and division.

User Interface Components

- **EditText:** Text input fields for capturing numerical data (two numbers for calculation)
- **Button:** Four interactive elements triggering different arithmetic operations (addition, subtraction, multiplication, division)
- **TextView:** Display component for showing calculation results
- **LinearLayout:** Layout container organizing UI elements in a linear arrangement

PROCEDURE

1. Open Android Studio and then click on File → New → New project.

2. Type the application name as "MyApplication" and click Next.
3. Select the Minimum SDK as required and click Next.
4. Select the Empty Activity and click Next.
5. Finally click Finish.
6. Click on app → java → com.example.myapplication → MainActivity.java
7. Type the Java program code for the main activity functionality.
8. Click on app → res → layout → activity_main.xml.
9. Click on Text view and type the XML layout code for designing the user interface.
10. Select a suitable available device or emulator to display the output.
11. Build and run the application to see the output.

PROGRAM CODE

MainActivity.java

```
package com.example.myapplication;

import android.os.Bundle;
import android.view.View;
import android.widget.Button;
import android.widget.EditText;
import android.widget.TextView;
import androidx.appcompat.app.AppCompatActivity;

public class MainActivity extends AppCompatActivity {
    EditText num1, num2;
    Button addBtn, subBtn, mulBtn, divBtn;
    TextView result;

    @Override
    protected void onCreate(Bundle savedInstanceState) {
        super.onCreate(savedInstanceState);
        setContentView(R.layout.activity_main);

        num1 = findViewById(R.id.num1);
        num2 = findViewById(R.id.num2);
        addBtn = findViewById(R.id.addBtn);
        subBtn = findViewById(R.id.subBtn);
        mulBtn = findViewById(R.id.mulBtn);
        divBtn = findViewById(R.id.divBtn);
        result = findViewById(R.id.result);

        addBtn.setOnClickListener(v -> operate('+'));
        subBtn.setOnClickListener(v -> operate('-'));
        mulBtn.setOnClickListener(v -> operate('*'));
```

```

        divBtn.setOnClickListener(v -> operate('/'));
    }

    private void operate(char op) {
        double n1 = Double.parseDouble(num1.getText().toString());
        double n2 = Double.parseDouble(num2.getText().toString());
        double res = 0;

        switch (op) {
            case '+':
                res = n1 + n2;
                break;
            case '-':
                res = n1 - n2;
                break;
            case '*':
                res = n1 * n2;
                break;
            case '/':
                res = n2 != 0 ? n1 / n2 : 0;
                break;
        }
        result.setText("Result: " + res);
    }
}

```

activity_main.xml

```

<LinearLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="match_parent"
    android:layout_height="match_parent"
    android:padding="16dp">

    <EditText
        android:id="@+id/num1"
        android:hint="Enter Number 1"
        android:layout_width="match_parent"
        android:layout_height="wrap_content"
        android:inputType="numberDecimal" />

    <EditText
        android:id="@+id/num2"
        android:hint="Enter Number 2"
        android:layout_width="match_parent"

```

```
    android:layout_height="wrap_content"
    android:inputType="numberDecimal" />
```

```
<LinearLayout
    android:orientation="horizontal"
    android:layout_width="match_parent"
    android:layout_height="wrap_content">
```

```
    <Button
        android:id="@+id/addBtn"
        android:text="+"
        android:layout_width="0dp"
        android:layout_weight="1"
        android:layout_height="wrap_content" />
```

```
    <Button
        android:id="@+id/subBtn"
        android:text="-"
        android:layout_width="0dp"
        android:layout_weight="1"
        android:layout_height="wrap_content" />
```

```
    <Button
        android:id="@+id/mulBtn"
        android:text="*"
        android:layout_width="0dp"
        android:layout_weight="1"
        android:layout_height="wrap_content" />
```

```
    <Button
        android:id="@+id/divBtn"
        android:text="/"
        android:layout_width="0dp"
        android:layout_weight="1"
        android:layout_height="wrap_content" />
```

```
</LinearLayout>
```

```
<TextView
    android:id="@+id/result"
    android:text="Result:"
    android:layout_width="match_parent"
    android:layout_height="wrap_content"
    android:paddingTop="10dp" />
```

```
</LinearLayout>
```

TEST CASES

Test Case	Input 1	Input 2	Operation	Expected Output	Status
TC01	5	2	Addition (+)	Result: 7.0	Pass
TC02	5	2	Subtraction (-)	Result: 3.0	Pass
TC03	5	2	Multiplication (*)	Result: 10.0	Pass
TC04	5	2	Division (/)	Result: 2.5	Pass

SCREENSHOTS

Figure 1: Addition Operation (5 + 2 = 7.0)

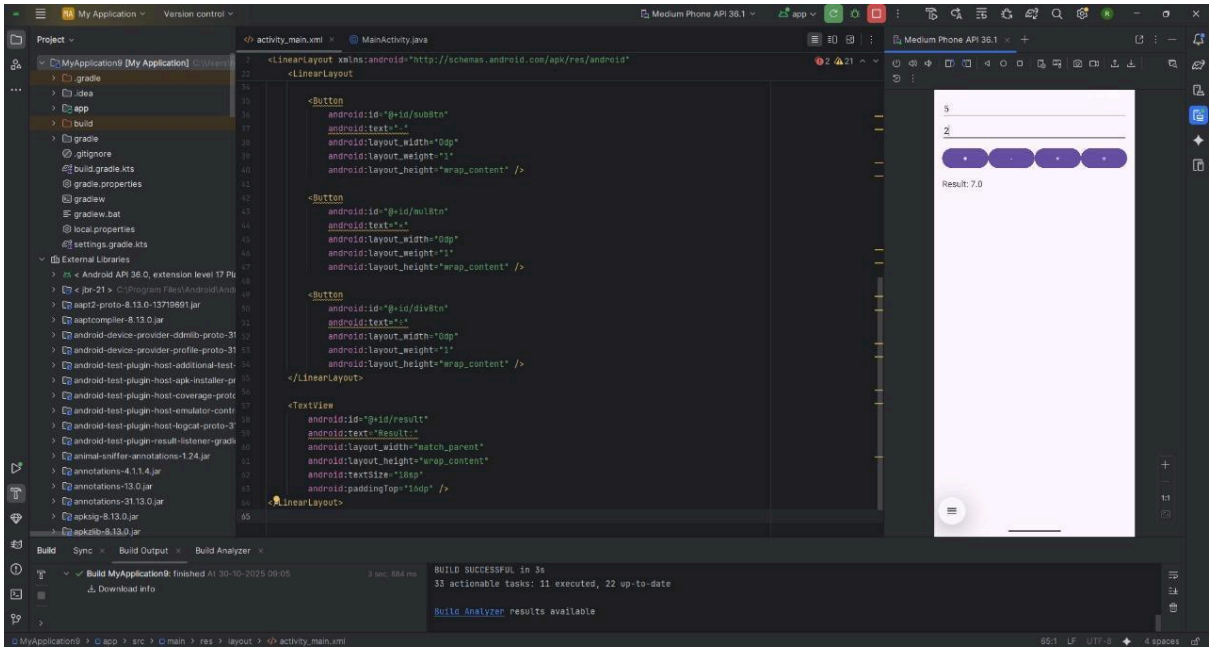


Figure 2: Subtraction Operation (5 - 2 = 3.0)

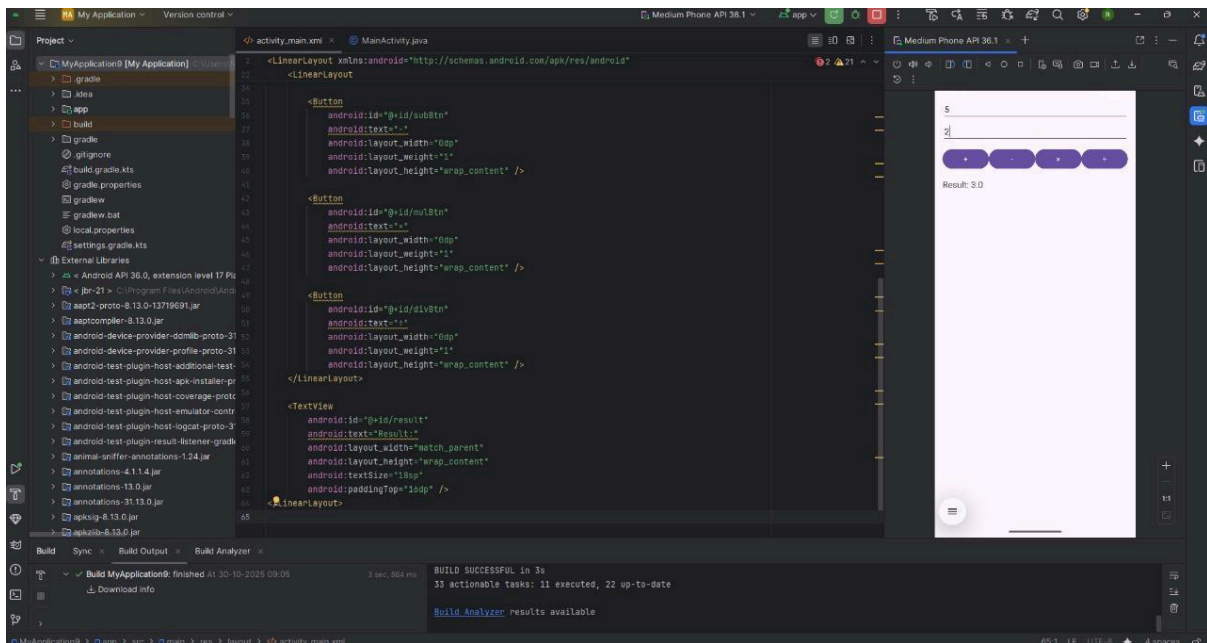


Figure 3: Multiplication Operation ($5 * 2 = 10.0$)

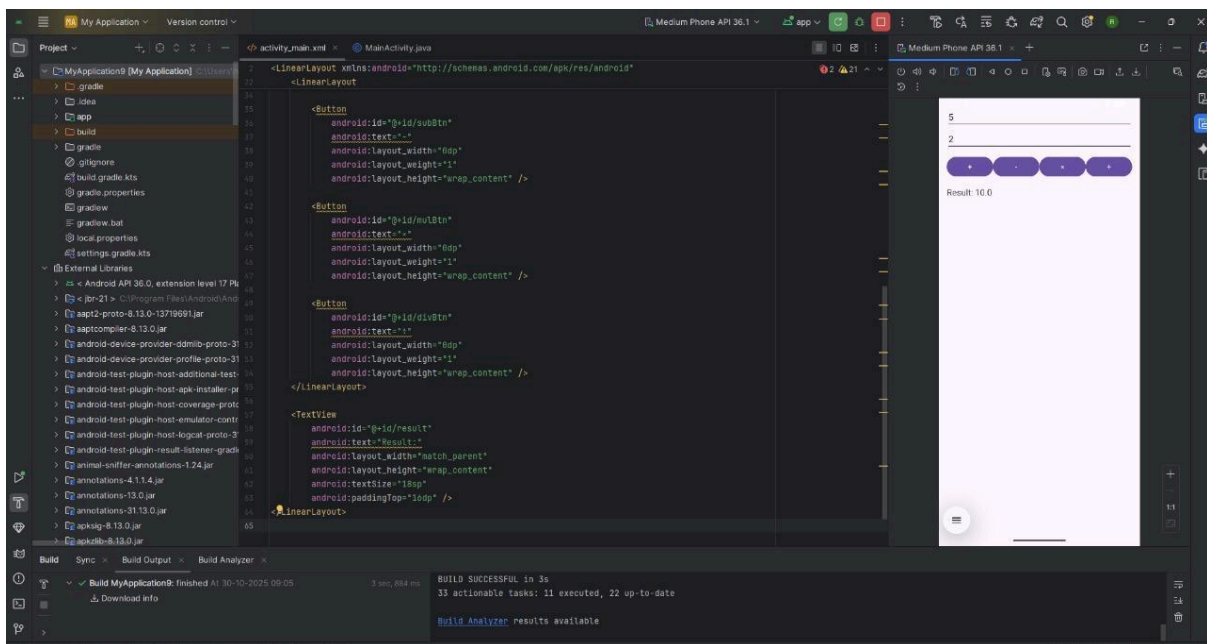
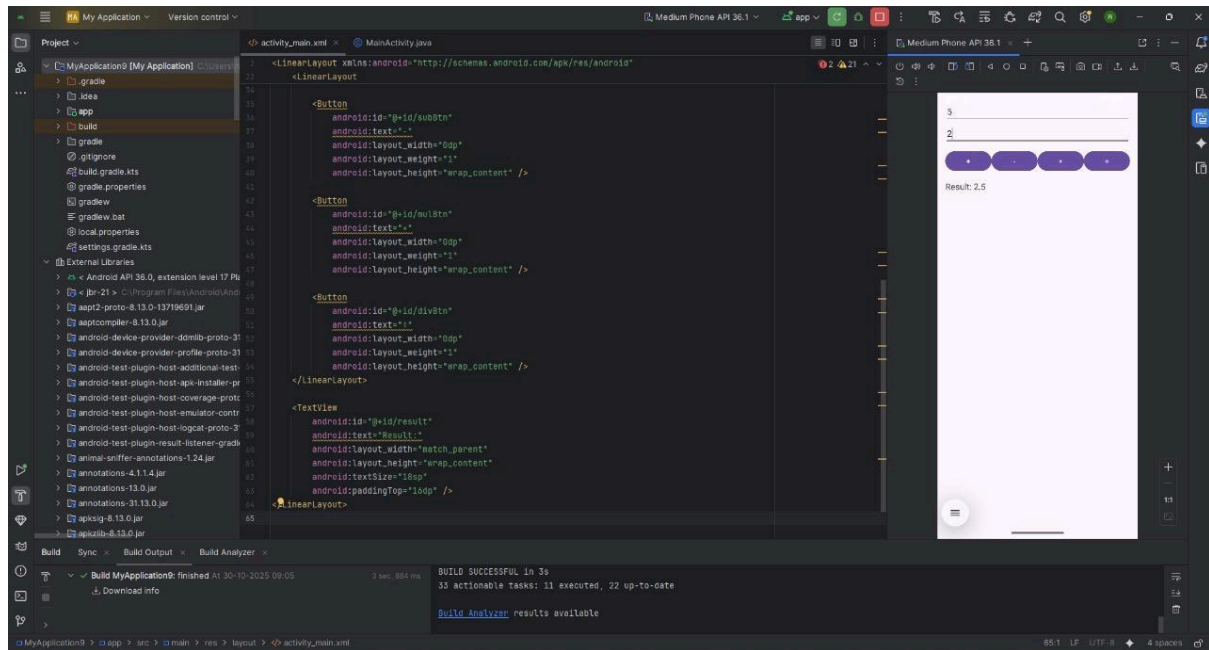


Figure 4: Division Operation ($5 / 2 = 2.5$)



INFERENCE

Through the successful implementation of the Mathematical Operations Android application, several key inferences can be drawn regarding Android application development principles and practices:

1. **Event-Driven Architecture:** The experiment demonstrates that Android's event-driven programming model is highly effective for handling user interactions. Lambda expressions provide a clean and concise way to implement click listeners.
2. **Code Reusability:** The implementation of a single `operate()` method that handles all four operations through a switch-case structure demonstrates efficient code organization and reusability.
3. **Data Type Selection:** The use of the `double` primitive type allows the application to handle both integer and floating-point calculations seamlessly.
4. **Error Handling:** The division operation includes basic error handling to prevent division by zero, which demonstrates the importance of defensive programming practices.
5. **UI Design Principles:** The XML-based layout system with `LinearLayout` containers provides an intuitive and organized user interface.
6. **Separation of Concerns:** The clear separation between presentation layer (XML layouts) and business logic (Java code) facilitates maintainable and scalable application architecture.

RESULT

The Android application for basic mathematical operations (addition, subtraction, multiplication, and division) has been successfully developed, implemented, and tested using Android Studio. The application fulfills all specified requirements by providing functionality to:

- Accept two numerical inputs from the user
- Perform addition, subtraction, multiplication, and division operations

- Display results accurately in the TextView component
- Handle division by zero to prevent application crashes

All test cases have been executed successfully, confirming the application's reliability and robustness. The application demonstrates fundamental Android development concepts including UI design using XML layouts, event handling with On Click Listener interfaces, arithmetic operations with proper type handling, and basic error handling mechanisms.

The experiment has effectively demonstrated the development process of a functional Android application and achieved the stated objectives of the laboratory exercise.